Supplementary Material for LinStabPy: compressible stability and resolvent analysis

Gabriel Y. R. Hamada[1,2], Ricardo A. S. Frantz[3]

[1] School of Mechanical Engineering, University of Campinas, Campinas, SP 13083-860, Brazil

[2] DynFluid, Arts et Métiers Paris / CNAM, 151 Bd de l'Hôpital, 75013 Paris, France

[3] Laboratory of Fluid Mechanics and Instabilities,
École Polytechnique Fédérale de Lausanne, CH-1015 Lausanne, Switzerland

S1 Scope of the Supplementary Material

This Supplementary Material contains the technical detail that supports the brief communication. Its role is fourfold: to record the governing equations and discrete-operator formulation implemented in LinStabPy; to document the SBP/curvilinear discretization, boundary treatment, filtering, and solver structure; to separate internal fidelity from external validation; and to provide a reproducibility and code map.

The brief communication is concise by design; the detailed derivations, verification protocol, benchmark-fidelity tables, solver-configuration structure, and harmonic-resolvent extension are collected here.

S1.1 Software context and repository provenance

LinStabPy builds on two public repositories. The `LinStab2D` repository [1, 2] is the direct source of the compressible stability/resolvent formulation around which the present package is organized: the five-variable compressible LNSE, structured and curvilinear grids, summation-by-parts finite differences, boundary-condition recipes, filtering, and the example family used for fidelity assessment. The `resolvent4py` repository [3–5] is the direct software reference for the operator-abstraction side: matrix-backed operators, operator products, projections, and the idea that several analysis workflows should share the same operator language rather than being implemented as isolated scripts.

Martini and Schmidt [2] document the upstream compressible MATLAB framework. Padovan and co-workers develop the operator and harmonic-resolvent formulation in the `resolvent4py` software paper [4] and in the harmonic-resolvent theory papers [6, 7], while Farghadan et al. [8] address scalable harmonic-resolvent computation.

Several adjacent packages occupy different positions in the software landscape. `BROADCAST` [9] and `dNami` [10] are broader nonlinear/compressible frameworks designed for wider CFD workflows; `BROADCAST` also includes stability/sensitivity capability through algorithmic differentiation. These packages address a broader scope than the one targeted here. `LightKrylov` [11] focuses on

Krylov-based numerics; see Frantz et al. [12] for a review of Krylov methods in fluid-dynamics applications.

The role of LinStabPy within that landscape is narrower. The package does not attempt to be a general nonlinear flow solver, nor does it generate tangent or adjoint operators by differentiating a full nonlinear time-marching code. Many users can obtain a nonlinear base or mean flow from their existing CFD or data-generation environment but lack a ready-to-use compressible linear operator with the needed thermodynamic, curvilinear, filtering, and boundary-condition detail. LinStabPy targets that gap: the user supplies the reference flow, and the package discretizes the corresponding linearized operator for stability and resolvent analysis, from prototyping to production-scale sparse solves.

The evidence hierarchy is kept explicit:

1. independent verification of the assembled discrete operator;
2. fidelity to the shipped Martini example family;
3. external validation against the Robinet/Song oblique SWBLI benchmark;
4. application-facing Case 1 artifacts for the current turbine-cascade workflow;
5. harmonic-resolvent and solver-extension structure.

Quantitative claims should be interpreted against this hierarchy.

S2 Governing equations and discrete-operator formulation

S2.1 Compressible linearized Navier–Stokes equations

We consider the compressible Navier–Stokes equations for a calorically perfect gas, linearized about a steady base flow $\bar{\mathbf{q}}(\mathbf{x})$. The flow state is decomposed as

$$\mathbf{q}(\mathbf{x}, t) = \bar{\mathbf{q}}(\mathbf{x}) + \epsilon\, \mathbf{q}'(\mathbf{x}, t), \qquad 0 < \epsilon \ll 1, \tag{1}$$

where the perturbation state vector in primitive variables is

$$\mathbf{q}' = [\rho',\, u',\, v',\, w',\, T']^\top. \tag{2}$$

Here ρ is density, (u, v, w) are the velocity components in the (x, y, z) directions, and T is temperature.

All quantities are non-dimensionalized by a reference length L^*, velocity U^*, density ρ^*, and temperature T^*, yielding three governing parameters: the Reynolds number $Re = \rho^* U^* L^*/\mu^*$, the Mach number $Ma = U^*/a^*$ with $a^* = \sqrt{\gamma R_g T^*}$, and the Prandtl number $Pr = \mu^* c_p/\kappa^*$. The equation of state for a perfect gas gives the non-dimensional pressure as

$$p = \frac{\rho\, T}{\gamma\, Ma^2}. \tag{3}$$

The dynamic viscosity follows Sutherland's law:

$$\mu(T) = \mu_{\text{ref}}\, \frac{T_{\text{ref}} + S_1}{T + S_1} \left(\frac{T}{T_{\text{ref}}}\right)^{3/2}, \qquad S_1 = \frac{110.4\ \text{K}}{T^*}. \tag{4}$$

The bulk viscosity follows the Stokes hypothesis, $\lambda = -\frac{2}{3}\mu$, and thermal conductivity is determined by Pr.

Following Martini and Schmidt [2], the linearized system is written in matrix form as

$$\mathbf{M}\,\frac{\partial \mathbf{q}'}{\partial t} = \left(\mathbf{A}_0 + \mathbf{A}_x D_x + \mathbf{A}_y D_y + \mathbf{A}_{xx} D_{xx} + \mathbf{A}_{yy} D_{yy} + \mathbf{A}_{xy} D_{xy}\right) \mathbf{q}', \tag{5}$$

where D_x, D_y, D_{xx}, D_{yy}, and D_{xy} denote the first and second partial-derivative operators, and the 5×5 coefficient matrices $\mathbf{A}_k$ depend on the base-flow variables and their gradients. The matrix $\mathbf{M}$ accounts for the density weighting of the primitive-variable momentum and energy equations; for Cartesian two-dimensional configurations it reduces to $\mathbf{M} = \mathrm{diag}(1, \bar{\rho}, \bar{\rho}, \bar{\rho}, \bar{\rho})$, while for axisymmetric flows it becomes lower-triangular to perform the conservative-to-primitive transformation. The explicit expressions for the coefficient matrices are taken from Martini and Schmidt [2]; the present Python implementation is a translation of their released MATLAB formulation.

To guard against errors propagated through the derivation chain (Mathematica → MATLAB → Python), the assembled operator is verified independently by symbolic re-derivation and by a complex-step Jacobian check against the full nonlinear residual; that verification protocol is documented in Section S4.

Base flow vs. mean flow. Equation (5) assumes that $\bar{\mathbf{q}}$ is a steady solution of the Navier–Stokes equations. In practice, $\bar{\mathbf{q}}$ may instead be a time-averaged (and possibly spanwise-averaged) field obtained from DNS or LES. Such a mean flow is not, in general, an exact NSE solution because the Reynolds-stress divergence is omitted. Linearization about a mean flow is nonetheless standard [13]; the operator retains the same structure, but the interpretation of the resulting spectra and gains must account for the missing nonlinear feedback.

Spanwise Fourier decomposition. The spanwise direction is assumed homogeneous, so perturbations are decomposed into Fourier modes:

$$\mathbf{q}'(x, y, z, t) = \hat{\mathbf{q}}(x, y, t)\, \mathrm{e}^{\mathrm{i}k_z z}, \tag{6}$$

where k_z is the spanwise wavenumber. Under this ansatz the z-derivative operators become algebraic multiplications ($D_z \to \mathrm{i}k_z$ and $D_{zz} \to -k_z^2$), reducing the problem to a two-dimensional one parameterized by k_z.

S2.2 Temporal stability analysis

Temporal stability analysis seeks the complex frequencies ω at which the linearized system admits exponentially growing or decaying solutions. With the normal-mode ansatz $\mathbf{q}'(\mathbf{x}, t) = \hat{\mathbf{q}}(\mathbf{x})\, \mathrm{e}^{-\mathrm{i}\omega t}$, substitution into (5) yields the generalized eigenvalue problem

$$\mathbf{L}\,\hat{\mathbf{q}} = \lambda\, \mathbf{M}\,\hat{\mathbf{q}}, \qquad \lambda = -\mathrm{i}\omega, \tag{7}$$

where $\mathbf{L}$ is the spatially discretized LNSE operator and $\mathbf{M}$ is the mass or conversion matrix. In the implementation, all factors of i are confined to the frequency-domain problem; temporally unstable modes therefore satisfy $\mathrm{Im}(\omega) > 0$.

For targeted extraction of eigenvalues near a shift σ, the shift-invert transformation is used:

$$(\mathbf{L} - \sigma\mathbf{M})^{-1}\mathbf{M}\,\mathbf{v} = \mu\,\mathbf{v}, \qquad \mu = \frac{1}{\lambda - \sigma}. \tag{8}$$

Each Krylov iteration of the outer eigensolver therefore requires one sparse linear solve with the shifted matrix $(\mathbf{L} - \sigma\mathbf{M})$. In the current implementation, including the Song benchmark runs reported below, the outer eigensolver is SLEPc Krylov–Schur and the shifted linear systems are solved directly with PETSc `KSP = PREONLY`, `PC = LU`, and MUMPS as the factorization backend. Thus the eigenvalue workflow is Krylov at the outer eigensolver level, but not iterative at the inner linear-solver level.

Adjoint eigenvalues. The adjoint operator in the $\mathbf{W}$-weighted inner product is

$$\mathbf{L}_{\mathbf{W}}^{+} = \mathbf{W}^{-1}\mathbf{L}^{H}\mathbf{W}, \tag{9}$$

where $\mathbf{W}$ is the diagonal weight matrix defined below for the Chu energy inner product.

S2.3 Spatial stability analysis

Spatial stability analysis seeks the complex streamwise wavenumber α for prescribed real frequency ω. Under the ansatz $\mathrm{e}^{\mathrm{i}\alpha x}$, downstream spatial growth corresponds to $\mathrm{Im}(\alpha) < 0$. Because the LNSE operator depends quadratically on α through the Floquet-shifted derivatives, the resulting problem is a quadratic eigenvalue problem:

$$(\mathbf{R}_0 + \mathrm{i}\omega\mathbf{I} + \alpha\,\mathbf{R}_1 + \alpha^2\,\mathbf{R}_2)\,\hat{\mathbf{q}} = \mathbf{0}, \tag{10}$$

where the matrices $\mathbf{R}_0$, $\mathbf{R}_1$, and $\mathbf{R}_2$ are reconstructed from three operator evaluations with finite-difference perturbations in α [14]. The QEP is then linearized to a generalized eigenvalue problem of doubled dimension through the usual companion form:

$$\mathbf{L}_c = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{R}_0 & \mathbf{0} \end{bmatrix}, \qquad \mathbf{R}_c = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{R}_1 & \mathbf{R}_2 \end{bmatrix}, \qquad \mathbf{L}_c\hat{\mathbf{v}} = \alpha\,\mathbf{R}_c\hat{\mathbf{v}}. \tag{11}$$

S2.4 Resolvent analysis

Resolvent analysis studies the amplification of harmonic forcing at frequency ω_f through the transfer operator

$$\mathbf{H}(\omega_f) = (-\mathrm{i}\omega_f\mathbf{M} - \mathbf{L})^{-1}. \tag{12}$$

For compressible flows, gains are interpreted in a weighted inner product based on the Chu disturbance-energy norm [15, 16],

$$E_{\mathrm{Chu}} = \frac{1}{2}\int_{\Omega}\left(\bar{\rho}(|u'|^2+|v'|^2+|w'|^2)+\frac{|p'|^2}{\gamma Ma^2\bar{T}}+\frac{\bar{\rho}}{\gamma(\gamma-1)Ma^2\bar{T}}|T'|^2\right)\mathrm{d}\Omega, \tag{13}$$

which leads to a diagonal discrete weight matrix $\mathbf{W}$ after quadrature and variable ordering:

$$\mathbf{W} = \mathrm{diag}(w_1,\ldots,w_n). \tag{14}$$

The dominant gains are obtained from the weighted singular-value problem

$$\mathbf{W}^{1/2}\mathbf{H}^H\mathbf{W}\mathbf{H}\mathbf{W}^{-1/2}\,\mathbf{v} = \sigma^2\mathbf{v}. \tag{15}$$

S2.5 Floquet-shifted operators

For streamwise-periodic settings, perturbations are written in Floquet form

$$\mathbf{q}'(x,y,t) = \hat{\mathbf{q}}(x,y)\,\mathrm{e}^{\mathrm{i}\alpha x-\mathrm{i}\omega t}, \tag{16}$$

so the streamwise derivatives are shifted as

$$D_x \to D_x + \mathrm{i}\alpha\mathbf{I}, \qquad D_{xx} \to D_{xx} + 2\mathrm{i}\alpha D_x - \alpha^2\mathbf{I}. \tag{17}$$

This is the operator-level bridge between the temporal, spatial, and streamwise-periodic formulations used throughout the package.

S2.6 Discrete operator assembly

After coordinate transformation and boundary treatment, the discrete LNSE operator $\mathbf{L}\in\mathbb{C}^{n\times n}$ with $n = 5N$ inherits the 5×5 block structure

$$\mathbf{L} = \begin{bmatrix} \mathbf{L}_{\rho\rho} & \mathbf{L}_{\rho u} & \mathbf{L}_{\rho v} & \mathbf{L}_{\rho w} & \mathbf{L}_{\rho T} \\ \mathbf{L}_{u\rho} & \mathbf{L}_{uu} & \mathbf{L}_{uv} & \mathbf{L}_{uw} & \mathbf{L}_{uT} \\ \mathbf{L}_{v\rho} & \mathbf{L}_{vu} & \mathbf{L}_{vv} & \mathbf{L}_{vw} & \mathbf{L}_{vT} \\ \mathbf{L}_{w\rho} & \mathbf{L}_{wu} & \mathbf{L}_{wv} & \mathbf{L}_{ww} & \mathbf{L}_{wT} \\ \mathbf{L}_{T\rho} & \mathbf{L}_{Tu} & \mathbf{L}_{Tv} & \mathbf{L}_{Tw} & \mathbf{L}_{TT} \end{bmatrix}. \tag{18}$$

For the strictly two-dimensional case $k_z = 0$, the spanwise-velocity equation decouples and the off-diagonal blocks involving w' vanish. For $k_z \neq 0$, continuity, pressure, and viscous terms couple w' to the remaining variables. On unmasked rectangular grids each $N\times N$ block is assembled efficiently with Kronecker products of the one-dimensional SBP operators; for masked grids a line-by-line sparse assembly is used with Fortran-order indexing. The operator is stored in compressed sparse column format with complex double precision before PETSc conversion.

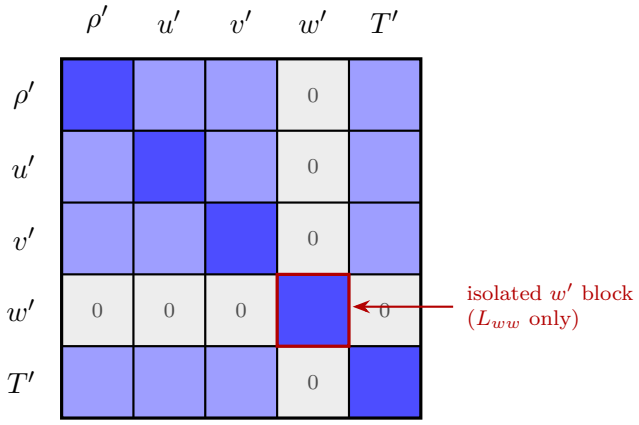


Figure S1: Schematic 5×5 block structure of the assembled LNSE operator $\mathbf{L}$ for the special case $k_z = 0$. For $k_z \neq 0$, the w'-coupling blocks become nonzero.

S3 Discretization: SBP, curvilinear terms, BCs, and filtering

S3.1 SBP finite differences

Spatial derivatives are approximated with summation-by-parts finite-difference operators [17]. In one dimension an SBP first-derivative operator is written as

$$D = H^{-1}Q, \qquad Q + Q^{\top} = B, \tag{19}$$

where H is a symmetric positive-definite quadrature matrix and B encodes the boundary terms. This discrete integration-by-parts structure is the basis for the boundary treatment and the weighted adjoint relations used later in the resolvent workflow. The implementation ships reference tables for order-2, 4, 6, and 8 SBP operators on structured meshes, and includes an automatic generator for arbitrary even order. Interior stencils are computed via the Fornberg algorithm [18], while boundary closures are determined from the accuracy conditions and the SBP constraint (19); the free parameters that arise for orders ≥ 6 are resolved by maximising the accuracy at the rows nearest the interior [19]. This makes the framework extensible to higher-order discretisations without manual coefficient transcription.

S3.2 Curvilinear coordinate transformation

Curvilinear and stretched meshes are treated through a mapping from computational coordinates (ξ, η) to physical coordinates (x, y) with Jacobian

$$J = x_\xi y_\eta - x_\eta y_\xi. \tag{20}$$

The physical derivatives are related to computational derivatives by

$$\partial_x = \frac{y_\eta \partial_\xi - y_\xi \partial_\eta}{J}, \qquad \partial_y = \frac{-x_\eta \partial_\xi + x_\xi \partial_\eta}{J}. \tag{21}$$

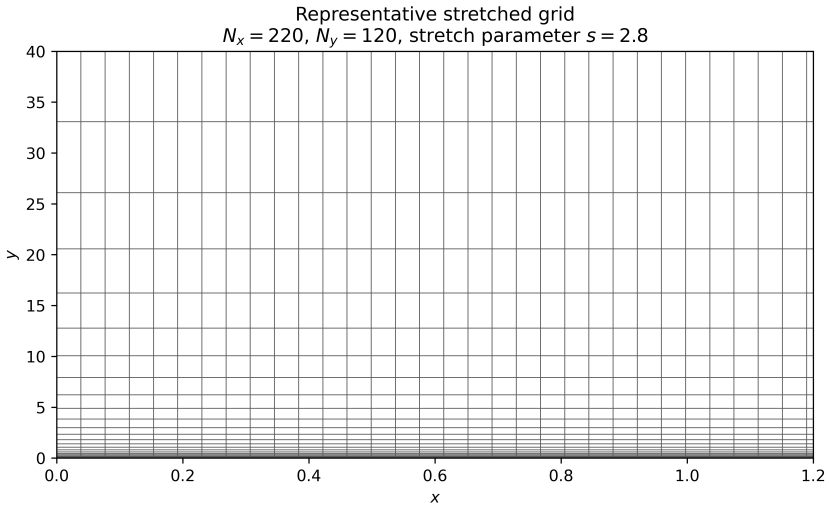


Figure S2: Representative stretched grid with wall-normal clustering controlled by the stretching parameter s.

For common one-dimensional wall-normal stretching, the expressions simplify to

$$D_y = \mathrm{diag}(1/y'_\eta)\, D_\eta, \qquad D_{yy} = \mathrm{diag}(1/{y'_\eta}^2)\, D_{\eta\eta} - \mathrm{diag}(y''_\eta/{y'_\eta}^3)\, D_\eta. \tag{22}$$

The package provides algebraic and hyperbolic-tangent stretching functions that cluster points near a wall while maintaining smoothness. A representative single-sided map is

$$y(\eta) = y_{\max} \frac{1 - \tanh[s(1-\eta)]/\tanh(s)}{1 + \tanh[s(1-\eta)]/\tanh(s)}, \tag{23}$$

where $s > 0$ controls the clustering intensity and $y_{\max}$ is the domain height.

S3.3 Boundary conditions

Boundary conditions are imposed through an algebraic overwrite. For homogeneous Dirichlet conditions on degree of freedom i, the generalized operator pair is overwritten as

$$A_{i,:} = 0, \qquad A_{:,i} = 0, \qquad M_{i,:} = 0, \qquad M_{:,i} = 0, \qquad A_{ii} = P_{\mathrm{bc}}, \qquad M_{ii} = 1, \tag{24}$$

while zero-normal-gradient conditions replace only the boundary row with the outward normal derivative,

$$L_{i,\cdot} = n_x D_{x,i_{\mathrm{mesh}},\cdot} + n_y D_{y,i_{\mathrm{mesh}},\cdot}. \tag{25}$$

Sponge layers for non-reflecting boundaries are implemented as a spatially varying diagonal damping term $\mathbf{L} \leftarrow \mathbf{L} - \mathrm{diag}(\boldsymbol{\sigma}_{\mathrm{sponge}})$.

This overwrite is not the classical SBP-SAT formulation. The parameter P_{bc} must balance boundary enforcement against shifted-system conditioning, because excessively large values can stall iterative Krylov convergence.

S3.4 Spatial filtering

High-frequency numerical oscillations are attenuated by a 10th-order implicit Padé-type filter in the framework of Lele [20] and Visbal and Gaitonde [21]. The filtered solution satisfies

$$\mathbf{M}_f \, \mathbf{q}_{\text{filt}} = \mathbf{A}_f \, \mathbf{q}, \tag{26}$$

where $\mathbf{M}_f$ is a symmetric tridiagonal matrix with unit diagonal and off-diagonal entries equal to the filter parameter α_{flt},

$$\mathbf{M}_f = \text{tridiag}(\alpha_{\text{flt}}, 1, \alpha_{\text{flt}}), \tag{27}$$

and $\mathbf{A}_f$ is the explicit filter operator. In two dimensions the filters are applied sequentially along the coordinate directions,

$$\mathbf{q}_{\text{filt}} = \mathbf{M}_{f,y}^{-1} \, \mathbf{M}_{f,x}^{-1} \, \mathbf{A}_{f,x} \, \mathbf{A}_{f,y} \, \mathbf{q}. \tag{28}$$

For resolvent analysis an adjoint-consistent filter pair is required so that the discrete adjoint relation is preserved in the weighted inner product.

S4 Verification of the discrete linearized operator

The coefficient matrices in (5) were originally derived with computer algebra, implemented in MATLAB, and then translated to Python. Agreement with the MATLAB reference is necessary for translation fidelity, but it cannot detect an error shared by both implementations. The assembled operator is therefore checked against two independent references that do not rely on the Martini derivation itself.

Complex-step Jacobian verification. The discrete nonlinear compressible Navier–Stokes residual $\mathbf{R}(\mathbf{q})$ is evaluated on the same grid and with the same SBP operators used for the linearized system. The exact Jacobian of $\mathbf{R}$ at the base flow $\bar{\mathbf{q}}$ is then computed column-by-column through the complex-step method [22, 23]:

$$\left.\frac{\partial R_i}{\partial q_j}\right|_{\bar{\mathbf{q}}} = \frac{\text{Im}[\mathbf{R}(\bar{\mathbf{q}} + \mathrm{i}\epsilon \, \mathbf{e}_j)]}{\epsilon}, \tag{29}$$

with $\epsilon \sim 10^{-30}$. Unlike finite differences, complex-step derivatives avoid subtractive cancellation and can approach machine precision provided the residual is implemented without non-analytic operations such as `abs` or `real`.

Symbolic re-derivation with SymPy. As a complementary check, the compressible Navier–Stokes equations are linearized from scratch with SymPy symbolic algebra. The base-flow and perturbation quantities are declared symbolically, the substitution $\mathbf{q} = \bar{\mathbf{q}} + \epsilon \mathbf{q}'$ is expanded in ϵ, and the coefficient matrices are extracted by grouping terms according to the derivative operators. This path is particularly useful for the Sutherland-viscosity derivative terms, which are algebraically dense and therefore prone to sign or factor mistakes.

Case	Scope	Primary automated audit metric	Status
Example 0: curvilinear derivative	protocol	$\max \lvert df_{\mathrm{py}} - df_{\mathrm{ml}} \rvert = 6.1 \times 10^{-14}$; Python and MATLAB derivatives agree to machine precision	PASS
Example 1: Blasius resolvent	direct	$\max \lvert \Delta\sigma \rvert / \sigma = 0.54\%$; SVD residual $\leq 2.3 \times 10^{-13}$	PASS
Example 2: jet resolvent	direct	$\max \lvert \Delta\sigma \rvert / \sigma = 0.20\%$; SVD residual $\leq 7.5 \times 10^{-13}$	PASS
Example 3: streaky boundary layer	temporal + spatial	Temporal eigenvalue residual $\leq 1.3 \times 10^{-11}$; spatial eigenvalue residual $\leq 5.2 \times 10^{-9}$	PASS
Example 4: cavity spectra/adjoints	direct	$\max \lvert \Delta\omega \rvert = 5.8 \times 10^{-6}$ (direct and adjoint); eigenvalue residual $\leq 5.0 \times 10^{-6}$	PASS
Example 5: paraboloid resolvent	direct	$\max \lvert \Delta\sigma \rvert / \sigma = 0.043\%$; SVD residual $\leq 8.5 \times 10^{-13}$	PASS

Table S1: Martini example-family parity audit. All six shipped examples satisfy the current quantitative closure criteria.

Verification status. The verification protocol produces two summary values: (i) the maximum relative discrepancy from the complex-step Jacobian check, and (ii) the symbolic agreement status for the coefficient sub-blocks. The Martini example-family audit reported next is kept distinct from these operator-level checks because it addresses translation fidelity rather than independent derivation verification.

S5 Fidelity to the released Martini example family

The external-reference audit against the released Martini MATLAB exports is the primary fidelity check for the shipped example family. The audit is regenerated by `./run_all_examples.sh`, which rewrites the Python example artifacts, extracts all available MATLAB parity bundles under `tools/parity/npz/`, and emits `tools/parity/reports/martini_parity_latest.json` plus `tools/parity/reports/martini_parity_table.csv`.

The example family is not a substitute for independent verification; it demonstrates that the Python translation reproduces the released MATLAB workflows to within the stated quantitative tolerances. The figures below show geometry, baseflow, mode, and spectrum artifacts; additional parity-comparison panels are available under `examples/`.

Example 0: curvilinear derivative validation. This example is a geometry/discretization protocol rather than a flow-physics case. A symmetric computational mesh is mapped to a parabolic physical domain, and the transformed derivative operators are checked against the analytical derivative of $\cos(x)$. Its role in the example family is to confirm that the curvilinear metrics and discrete derivative operators behave as intended before they are embedded in the full LNSE assembly.

Example 1: Blasius boundary-layer resolvent. The first flow case is a compressible flat-plate boundary layer. The baseflow panel shows the expected streamwise acceleration from the wall to the freestream, while the resolvent summary highlights forcing/response structures concentrated in the boundary-layer shear region. This case probes the convective-amplifier behavior of the discretized operator in a canonical wall-bounded shear flow.

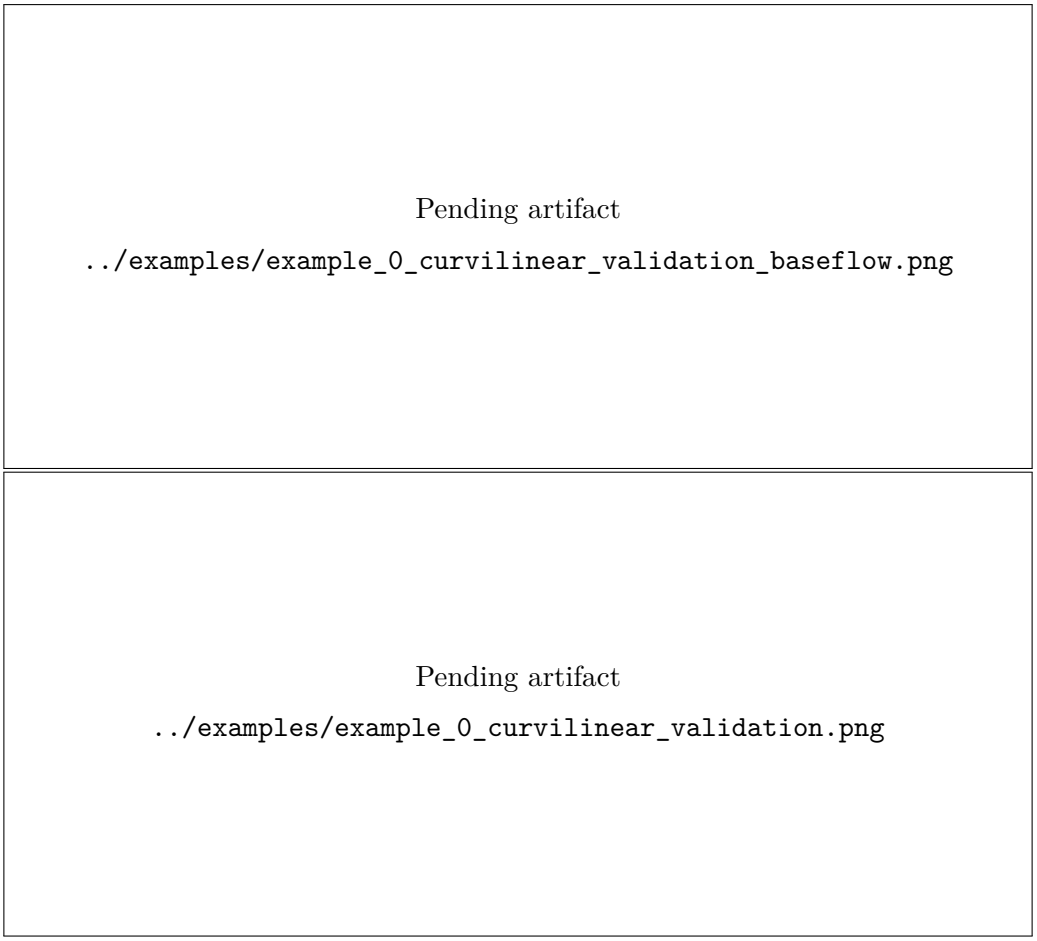


Figure S3: Example 0 curvilinear-derivative protocol: mesh/geometry view and derivative-verification summary.

Example 2: axisymmetric jet resolvent. This case moves to an axisymmetric jet base flow. The baseflow panel makes the jet core and surrounding shear layer explicit, and the resolvent summary shows the leading structures supported by that shear layer. The resolvent analysis again targets input–output amplification, now in a free-shear-flow setting with axisymmetric geometry and radial weighting in the compressible energy norm.

Example 3: streaky boundary-layer temporal and spatial analysis. This example introduces a streak-modulated boundary-layer base state and uses the same discretization machinery for both temporal and spatial analyses. The baseflow panel shows the streak modulation, the temporal panel highlights the leading disturbance structure at fixed streamwise wavenumber, the spatial panel shows the corresponding downstream-growing/decaying structure at fixed frequency, and the spectral panel summarizes the two spectra together. The physical interpretation is secondary instability of a streaky boundary layer rather than single-frequency resolvent amplification alone.

Example 4: cavity direct and adjoint global modes. The cavity case is the global-stability example in the family. The baseflow panel shows the recirculating cavity region and overlying shear layer. The mode panels then separate direct and adjoint structures. Their

joint interpretation is standard in global stability: the direct mode indicates where the global structure is expressed, while the adjoint mode indicates where the flow is most receptive to forcing.

Example 5: paraboloid resolvent. The final example returns to resolvent analysis, but on a strongly curved axisymmetric geometry obtained through a paraboloid mapping. The baseflow panel shows the resulting speed field over the mapped geometry, while the summary panel shows the leading forcing/response structures. Its role in the example family is to show that the same operator/discretization machinery extends to curved geometries without reducing the workflow to a Cartesian special case.

S6 External validation and current case-family status

S6.1 Oblique SWBLI benchmark: Robinet and Song

As an external benchmark, the project uses the oblique shock-wave/laminar-boundary-layer interaction studied by Robinet [24] and revisited with DNS confirmation by Song and Hao [25]. The configuration is a standard compressible global stability benchmark: the leading eigenvalue, the instability threshold, and the associated global mode provide a reference that is independent of the internal Martini example family. The current Python driver and reference data live under `validation/song_2023/`.

The comparison targets are the leading eigenvalue, the threshold value reported in the source literature, and the associated mode or spectrum overlays. Normalization conventions are stated alongside the comparison tables.

Validation status. After correcting the input-data layout convention (the saved MATLAB-exported fields use `(x_index, y_index)` ordering, which must be transposed before entering the `(ny, nx)` mesh path), both the SciPy-assembled and PETSc-assembled operators produce consistent eigenvalue spectra under direct LU/MUMPS factorization. On a reduced grid, the two assembly backends match exactly (mean nearest-MATLAB eigenvalue distance 1.37×10^{-2}, leading-mode growth rate $\mathrm{Im}(\omega_1) = +0.0605$). On the full 599×301 grid with the default filtered baseflow and an over-extraction strategy (24 modes requested, 20 nearest to target retained), the 20-mode comparison gives a mean nearest-MATLAB distance of 1.16×10^{-2} and a maximum of 3.01×10^{-2}. The leading eigenvalue is $\mathrm{Im}(\omega_1) = +0.0573$, compared to the MATLAB reference $+0.0589$ ($\approx 3\%$ difference). Only three matched pairs exceed 2×10^{-2}, and the previously observed deep-damped outlier pair is resolved by the enlarged extraction window.

Figure S8 shows the current Song summary artifact: a temporal spectrum panel with Python, MATLAB, and Song original eigenvalues overlaid, and side-by-side Python/MATLAB mode shapes for a steady (Mode 1) and an oscillatory (Mode 2) eigenmode. The Python eigenvectors are phase-aligned to MATLAB via a complex least-squares projection before plotting, so that the signed contour levels are directly comparable. All current Song stability results in this manuscript use the same solver stack: SLEPc Krylov–Schur as the outer eigensolver, shift-invert for spectral targeting, and direct LU factorization with MUMPS for the inner shifted solves.

Workflow	Grid	$n = 5N$	Leading output	Wall time [s]	Peak mem [GB]
Temporal EVP	$N_x \times N_y$	pending	$\omega_1 =$ pending	pending	pending
Resolvent	$N_x \times N_y$	pending	$\sigma_1 =$ pending	pending	pending

Table S2: Case 1 reporting template. Numerical entries are placeholders pending completion of the reproducible artifact pipeline.

S6.2 Application-facing Case 1 status

The current turbine-cascade workflow based on Lui et al. [26] plays a different role from the Robinet/Song benchmark. It is not used as the external-validation anchor; rather, it is the application case in which the same operator, solver, and resolvent infrastructure is applied to the project's target configuration. Because the supplied reference state is a mean flow, the interpretation of the resulting spectra and gains follows the mean-flow caveat stated in Section S2.1.

S7 Harmonic-resolvent formulation and extension

Classical resolvent analysis linearizes about a steady base or mean flow. When the dominant dynamics involve a time-periodic reference state or strong cross-frequency coupling, a steady-base resolvent may not capture the full amplification structure. This is the motivation for the harmonic-resolvent framework developed by Padovan et al. [6] and extended to subharmonic settings by Padovan and Rowley [7].

The core construction is still operator-based. Let the time-periodic base flow have fundamental frequency ω_0, and expand the perturbation on a finite harmonic set. The resulting harmonic operator has the schematic block form

$$\mathbf{L}_{\mathrm{harm}} = \begin{bmatrix} \ddots & \vdots & \ddots \\ \cdots & \mathbf{L}_0 - \mathrm{i}k\omega_0\mathbf{I} & \cdots \\ \ddots & \vdots & \ddots \end{bmatrix}, \tag{30}$$

with off-diagonal blocks carrying the cross-frequency coupling induced by the periodic base flow. Classical resolvent is therefore recovered as the steady-base limit of the broader harmonic formulation.

For LinStabPy, the main architectural implication is that harmonic resolvent should not be treated as an unrelated downstream script. It is another operator-level analysis that reuses the same physics-specific assembly while extending the operator to multiple harmonics. This motivates an explicit operator layer in the software design and a solver layer that can host direct, iterative, shell-based, or randomized harmonic workflows without changing the physical assembly interface.

In practice, harmonic-resolvent calculations require explicit choices for the retained harmonic set, truncation level, weighting strategy, and the reference state itself (periodic base flow versus mean-based surrogate). These choices are exposed as configuration parameters in the implementation.

S8 Solver configurations, preconditioners, and performance

S8.1 Solver architecture

The dominant computational cost of the analyses described above is repeated solution of large sparse shifted non-Hermitian systems: $(\mathbf{L} - \sigma\mathbf{M})\mathbf{x} = \mathbf{b}$ for eigenvalue problems and $(\mathbf{L} + \mathrm{i}\omega\mathbf{M})\mathbf{x} = \mathbf{b}$ for resolvent analysis. PETSc provides a unified interface for sparse matrices, iterative or direct linear solvers, and preconditioning, while SLEPc supplies the eigensolver and spectral transformation layers [27, 28]. The current default path assembles the operator in SciPy sparse format and converts it once to a PETSc AIJ matrix.

S8.2 Direct and iterative strategies

Two solver strategies are exposed. The *direct path* is a MUMPS sparse-direct baseline used for both eigenvalue problems and resolvent analysis:

- **Direct solver:** factor the shifted sparse matrix once, then apply forward/back substitutions for each right-hand side.

For the current Song stability calculations, this direct path appears inside the SLEPc shift-invert eigensolver: the outer algorithm is Krylov–Schur, while each shifted linear system is handled by direct LU/MUMPS. The *iterative path* is a Krylov/preconditioner route currently used only for the resolvent linear solves:

- **Iterative solver:** build successive approximations using Krylov matrix-vector products and a preconditioner until the residual tolerance is met.

In the current implementation, the iterative route is built around ASM/domain-decomposition preconditioners; for the algorithmic background of these methods, see Dolean et al. [29].

Remark: iterative inner solvers for eigenvalue problems. The code infrastructure allows an iterative KSP to replace the direct solver inside the SLEPc shift-invert spectral transformation. Systematic testing on the Martini example family has shown that lightweight preconditioners (ASM, block-Jacobi, ILU, Jacobi) diverge on all tested cases, and that GMRES with an exact LU preconditioner is the only iterative configuration that converges on the reduced Example 3 and Example 4 operators. Because even the LU- preconditioned iterative path has not yet been validated on harder cases, the eigenvalue path uses only the direct solver at present. See Section S8.3 for a discussion of the distinction between inner and outer Krylov strategies and the role of shifted-system conditioning.

The solver-selection workflow is intentionally explicit:

1. assemble and validate the operator on the target grid;
2. use the direct path (PREONLY + LU/MUMPS) for all eigenvalue problems;
3. for resolvent analyses, use the direct path as the reference, then switch to the iterative path when memory or distribution constraints make direct factorization impractical;
4. compare iterative resolvent outputs against a nearby direct-path reference case before scaling up;
5. reject production results when inner linear solves fail their prescribed tolerances.

Criterion	Direct path: MUMPS sparse-direct	Iterative path: preconditioned Krylov
Core mechanism	Sparse LU factorization once, then triangular solves for each right-hand side.	Iterative residual reduction with matrix-vector products and preconditioner applications.
Robustness	High robustness for difficult non-normal shifted systems.	Dependent on preconditioner quality and shift conditioning.
Memory profile	Higher memory demand due to fill-in and factor storage.	Lower memory footprint because no global LU factors are formed.
Setup cost	Potentially high one-time factorization cost.	Lower setup cost, but solve cost depends on iteration count.
Repeated solves	Efficient after factorization, especially for many solves at fixed shift.	Cost scales with the inner-iteration history and can vary across shifts/frequencies.
Current scope	Used for all eigenvalue problems and as the resolvent default.	Used for resolvent analyses only; not yet validated for the shift-invert eigensolver inner loop.

Table S3: Comparison of the two solver strategies. The iterative path is currently used only for resolvent analyses; see the remark in Section S8.2 regarding eigenvalue-problem limitations.

S8.3 Matrix-forming versus time-stepper Krylov strategies

A point of frequent confusion in the stability-analysis literature is the role of Krylov methods. The term "Krylov" appears in two distinct contexts that share mathematical foundations but differ in their numerical requirements and failure modes. Because LinStabPy is a matrix-forming code, this distinction directly determines the solver architecture.

Outer Krylov: the time-stepper approach. In the time-stepper formulation [12, 30, 31], the linearized operator $\mathbf{L}$ is never assembled. Instead, the action of the propagator $e^{\mathbf{L}\Delta t}$ (or an approximation to it) is evaluated by advancing the linearized equations one time step. The Arnoldi algorithm builds a Krylov basis $\{\mathbf{v}, e^{\mathbf{L}\Delta t}\mathbf{v}, (e^{\mathbf{L}\Delta t})^2\mathbf{v}, \ldots\}$ and extracts the dominant eigenvalues of the propagator, which correspond to the least-stable temporal modes [32]. No matrix is stored, no system is inverted, and the Arnoldi recurrence operates on the smooth output of the time-stepper. This is the approach used by spectral-element codes such as Nek5000 and Nektar++, and by operator-free resolvent tools such as `LightKrylov` [11].

Inner Krylov: iterative shift-invert. When a matrix-forming code needs interior eigenvalues near a target σ, the standard technique is the shift-invert spectral transformation: the eigensolver (Krylov–Schur in SLEPc) repeatedly applies $(\mathbf{L} - \sigma\mathbf{I})^{-1}$ to Arnoldi vectors [28, 32]. Each application requires solving the shifted linear system $(\mathbf{L} - \sigma\mathbf{I})\mathbf{x} = \mathbf{b}$. With a direct solver (LU/MUMPS) this reduces to forward/back substitutions using a precomputed factorization. If instead an iterative Krylov method (e.g. GMRES) is used for these inner solves, it must

Layer	Option pattern	Default role	Why chosen	When to change
SLEPc	ST = SINVERT	Interior-target extraction	Maps target eigenvalues to dominant transformed eigenpairs	Change shift σ when the target region moves
PETSc KSP/PC	PREONLY + LU(MUMPS)	EVP and resolvent direct inner solve	Necessary for the non-normal shifted LNSE systems encountered in practice	Replace only after validating an iterative alternative on the target case
PETSc KSP/PC	FGMRES + ASM	Resolvent iterative solve	Flexible Krylov route for variable preconditioning quality	Adjust overlap or preconditioner family when convergence stalls
Conv. guard	`ksp_error_if_not_converged`	Hard failure semantics	Prevents polluted outer spectra from unconverged inner solves	Do not disable for production claims

Table S4: Solver-option rationale for the current PETSc/SLEPc configuration layer.

invert the shifted operator to high accuracy at every outer Arnoldi step. The current Song runs use the former configuration: outer Krylov–Schur with direct LU/MUMPS inner solves, not GMRES/ASM or another iterative inner KSP.

Why the inner solve is demanding for compressible LNSE. The shifted temporal operator $i\mathbf{L} - \sigma\mathbf{I}$ inherits the non-normality of the linearized Navier–Stokes equations. For compressible flows, the LNSE operator couples five thermodynamic variables with disparate scales and supports convective non-normality [33, 34] as well as continuous-spectrum modes near the real axis. When the spectral target σ lies near the continuous spectrum or a cluster of eigenvalues, the shifted operator can become poorly conditioned [35, 36].

The practical consequences were observed in the Martini example family. On reduced grids of Example 3 (streaky boundary layer) and Example 4 (cavity), a systematic sweep of ten Krylov/preconditioner configurations found that only GMRES with an exact LU preconditioner (`gmres + lu`, right-preconditioned) converges; all lightweight preconditioners (ASM, block-Jacobi, ILU, Jacobi, fieldsplit) diverged (`KSP_DIVERGED_ITS`, 20 000 iterations). This indicates that the inner shifted solve requires a preconditioner that closely approximates the inverse of the shifted operator — local or incomplete approximations are insufficient for the non-normal compressible LNSE.

For the direct path (`PREONLY` + LU/MUMPS), the outer Krylov–Schur iteration checks only the *eigenpair* residual, not the inner linear-system residual. The direct solve therefore succeeds even on shifted systems where the inner-solve accuracy would be marginal for an iterative method, because the outer eigensolver tolerates moderate perturbations in the search directions.

Implications for solver selection. The distinction between outer and inner Krylov roles leads to a practical guideline:

- *Matrix-forming codes* (including LinStabPy): use direct factorization for the shift-invert inner solves. The formed matrix enables exact factorization, which decouples the outer

		Serial, direct LU/MUMPS			
Order	DOF ($5N$)	Assemble [s]	Solve [s]	Total [s]	Peak [GB]
fd2	901 495	pending	pending	pending	pending
fd4	901 495	pending	pending	pending	pending
fd6	901 495	pending	pending	pending	pending
fd8	901 495	pending	pending	pending	pending

Table S5: Song 2023 oblique SWBLI benchmark on the 599×301 grid: assembly and eigensolve cost versus finite-difference order. All runs are serial (1 MPI rank) with PETSc local assembly and MUMPS direct factorisation. The DOF count is constant because all orders share the same grid; only the stencil bandwidth changes.

eigenvalue convergence from inner-solve accuracy. This is the default and currently the only validated EVP path.

- *Time-stepper / matrix-free codes*: use the outer Krylov approach on the propagator. No inner linear system is formed or inverted; the non-normality is encoded in the time-stepper output, which the outer Arnoldi handles without shift-invert.
- *Resolvent analysis*: both paths are viable. The resolvent requires solving a single shifted system $(\mathbf{L} + \mathrm{i}\omega\mathbf{I})\mathbf{x} = \mathbf{b}$ rather than embedding the solve inside an outer eigenvalue loop. The accuracy requirement is therefore that of a one-shot linear solve, which is less demanding than the repeated inner solves needed by shift-invert. Iterative Krylov with domain-decomposition preconditioning (FGMRES + ASM) may therefore succeed for resolvent even when it fails for eigenvalue problems.

These observations are consistent with the broader experience reviewed by Theofilis [37] and Frantz et al. [12]: the choice between matrix-forming and time-stepper approaches is not simply a matter of preference but determines which solver strategies are available and where non-normality enters the computation.

S8.4 Performance structure

Performance reporting is organized around wall time, peak memory, solver configuration, and hardware metadata.

S9 Reproducibility and code map

The supplement keeps the paper-to-code map explicit so that each main equation or workflow can be traced to the implementation entry point.

Two command paths are kept separate on purpose.

For the external-reference Martini audit, the canonical command is

```
./run_all_examples.sh
```

which regenerates the comparison-eligible example outputs and refreshes the JSON/CSV audit reports under `tools/parity/reports/`.

For the current Case 1 workflow, the repository command path remains the case-specific driver under `results/case1/`. The paper-facing artifacts should be sourced from that workflow

Eq.	Description	File	Entry point
(5)	LNSE matrix form	`operators/coefficients.py`	`LNSECoefficients`
(7)–(8)	Temporal EVP	`solvers/eigenvalue.py`	`temporal_eigenvalues`
(9)	Adjoint operator	`solvers/eigenvalue.py`	`adjoint_eigenvalues`
(10)–(11)	Spatial EVP / companion form	`operators/spatial.py`	`build_spatial_operator`
(15)	Resolvent SVD	`solvers/resolvent.py`	`resolvent_analysis`
(13)–(14)	Chu energy norm	`operators/energy_norm.py`	`build_energy_norm`
(17)	Floquet shift	`operators/assembly.py`	`assemble_lnse`
(26)–(28)	Implicit filter	`mesh/filters.py`	`build_filter`

Table S6: Mapping of the main supplementary equations to the implementation entry points under `src/linstab2d/`.

rather than copied into the supplement by hand.

Code availability. LinStabPy is available at `https://github.com/ricardofrantz/linstabpy` as the Python package `linstabpy`; release DOI and snapshot metadata will be added in the final submission package.

References

[1] LinStab2D repository. `https://github.com/LinStab2D/LinStab2D`, 2026. GitHub repository, accessed 2026-03-13.

[2] Eduardo Martini and Oliver Schmidt. Linstab2D: stability and resolvent analysis of compressible viscous flows in MATLAB. *Theoretical and Computational Fluid Dynamics*, 38(5):665–685, 2024. doi: 10.1007/s00162-024-00706-0.

[3] resolvent4py repository. `https://github.com/albertopadovan/resolvent4py`, 2026. GitHub repository, accessed 2026-03-13.

[4] Alberto Padovan, Vishal Anantharaman, Clarence W. Rowley, Blaine Vollmer, Tim Colonius, and Daniel J. Bodony. Resolvent4py: An open-source package for stability, resolvent and modal decomposition analyses in python. *SoftwareX*, 31:102286, 2025. doi: 10.1016/j.softx.2025.102286.

[5] Alberto Padovan. resolvent4py example gallery: Cgl. `https://albertopadovan.github.io/resolvent4py/auto_examples/cgl/index.html`, 2026. Accessed 2026-03-01.

[6] Alberto Padovan, Samuel E. Otto, and Clarence W. Rowley. Analysis of amplification mechanisms and cross-frequency interactions in nonlinear flows via the harmonic resolvent. *Journal of Fluid Mechanics*, 900:A14, 2020. doi: 10.1017/jfm.2020.497.

[7] Alberto Padovan and Clarence W. Rowley. Analysis of the dynamics of subharmonic flow structures via the harmonic resolvent: application to vortex pairing in an axisymmetric jet. *Physical Review Fluids*, 7(7):073903, 2022. doi: 10.1103/PhysRevFluids.7.073903.

[8] Ali Farghadan, Junoh Jung, Rutvij Bhagwat, and Aaron Towne. Efficient harmonic resolvent analysis via time stepping. *Theoretical and Computational Fluid Dynamics*, 38(3):331–353, 2024. doi: 10.1007/s00162-024-00694-1.

[9] Arthur Poulain, Cédric Content, Denis Sipp, Georgios Rigas, and Eric Garnier. BROADCAST: A high-order compressible CFD toolbox for stability and sensitivity using algorithmic differentiation. *Computer Physics Communications*, 283:108557, 2023. doi: 10.1016/j.cpc.2022.108557.

[10] Nicolas Alferez, Emile Touber, Stephen Winn, and Yussuf Ali. dnami: a framework for solving systems of balance laws using explicit numerical schemes on structured meshes. `https://doi.org/10.5281/zenodo.6720593`, 2022. Zenodo software release.

[11] J. Simon Kern, Ricardo S. Frantz, and Jean-Christophe Loiseau. LightKrylov: Lightweight implementation of Krylov subspace techniques in modern Fortran. *Journal of Open Source Software*, 11(117):9623, 2026. doi: 10.21105/joss.09623.

[12] Ricardo A. S. Frantz, Jean-Christophe Loiseau, and Jean-Christophe Robinet. Krylov methods for large-scale dynamical systems: Application in fluid dynamics. *Applied Mechanics Reviews*, 75(3):030802, 2023. doi: 10.1115/1.4056808.

[13] Clément Mettot, Denis Sipp, and Hervé Bézard. Computation of eigenvalue sensitivity to base flow modifications in a discrete framework: Application to open-loop control. *Journal of Computational Physics*, 269:234–258, 2014. doi: 10.1016/j.jcp.2014.03.022.

[14] Mujeeb R. Malik. Numerical methods for hypersonic boundary layer stability. *Journal of Computational Physics*, 86(2):376–413, 1990.

[15] Boa-Teh Chu. On the energy transfer to small disturbances in fluid flow (Part I). *Acta Mechanica*, 1(3):215–234, 1965.

[16] Ardeshir Hanifi, Peter J. Schmid, and Dan S. Henningson. Compressible stability of growing boundary layers using the PSE and comparison with experiment and linear stability theory. *Theoretical and Computational Fluid Dynamics*, 8(3):219–235, 1996.

[17] Ken Mattsson and Jan Nordström. Summation by parts operators for finite difference approximations of second derivatives. *Journal of Computational Physics*, 199(2):503–540, 2004.

[18] Bengt Fornberg. Generation of finite difference formulas on arbitrarily spaced grids. *Mathematics of Computation*, 51(184):699–706, 1988.

[19] Bo Strand. Summation by parts for finite difference approximations for d/dx. *Journal of Computational Physics*, 110(1):47–67, 1994.

[20] Sanjiva K. Lele. Compact finite difference schemes with spectral-like resolution. *Journal of Computational Physics*, 103(1):16–42, 1992.

[21] Miguel R. Visbal and Datta V. Gaitonde. On the use of higher-order finite-difference schemes on curvilinear and deforming meshes. *Journal of Computational Physics*, 181(1): 155–185, 2002.

[22] William Squire and George Trapp. Using complex variables to estimate derivatives of real functions. *SIAM Review*, 40(1):110–112, 1998.

[23] Joaquim R. R. A. Martins, Peter Sturdza, and Juan J. Alonso. The complex-step derivative approximation. *ACM Transactions on Mathematical Software*, 29(3):245–262, 2003.

[24] Jean-Christophe Robinet. Bifurcations in shock-wave/laminar-boundary-layer interaction: global instability approach. *Journal of Fluid Mechanics*, 579:85–112, 2007. doi: 10.1017/S0022112007005095.

[25] Ziming Song and Jiaao Hao. Global instability of the interaction between an oblique shock and a laminar boundary layer. *Physics of Fluids*, 35(8), 2023. doi: 10.1063/5.0160648.

[26] Hugo F. S. Lui, Tulio R. Ricciardi, William R. Wolf, James Braun, Iman Rahbari, and Guillermo Paniagua. Unsteadiness of shock-boundary layer interactions in a Mach 2.0 supersonic turbine cascade. *Physical Review Fluids*, 7(9):094602, 2022. doi: 10.1103/PhysRevFluids.7.094602.

[27] Satish Balay et al. Petsc/tao users manual. *Argonne National Laboratory*, 2025. ANL-95/11 - Revision 3.24.

[28] Vicente Hernandez, Jose E. Roman, and Vicente Vidal. SLEPc: A scalable and flexible toolkit for the solution of eigenvalue problems. *ACM Transactions on Mathematical Software*, 31(3):351–362, 2005.

[29] Victorita Dolean, Pierre Jolivet, and Frédéric Nataf. *An Introduction to Domain Decomposition Methods: Algorithms, Theory, and Parallel Implementation.* SIAM, 2015.

[30] W. S. Edwards, Laurette S. Tuckerman, Richard A. Friesner, and Danny C. Sorensen. Krylov methods for the incompressible Navier–Stokes equations. *Journal of Computational Physics*, 110(1):82–102, 1994.

[31] Dwight Barkley, Hugh M. Blackburn, and Spencer J. Sherwin. Direct optimal growth analysis for timesteppers. *International Journal for Numerical Methods in Fluids*, 57(9): 1435–1458, 2008.

[32] Richard B. Lehoucq, Danny C. Sorensen, and Chao Yang. *ARPACK Users' Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods.* SIAM, 1998.

[33] Lloyd N. Trefethen, Anne E. Trefethen, Satish C. Reddy, and Tobin A. Driscoll. Hydrodynamic stability without eigenvalues. *Science*, 261(5121):578–584, 1993.

[34] Peter J. Schmid. Nonmodal stability theory. *Annual Review of Fluid Mechanics*, 39:129–162, 2007.

[35] Lloyd N. Trefethen and Mark Embree. *Spectra and Pseudospectra: The Behavior of Nonnormal Matrices and Operators*. Princeton University Press, 2005.

[36] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, 2 edition, 2003.

[37] Vassilis Theofilis. Global linear instability. *Annual Review of Fluid Mechanics*, 43:319–352, 2011.

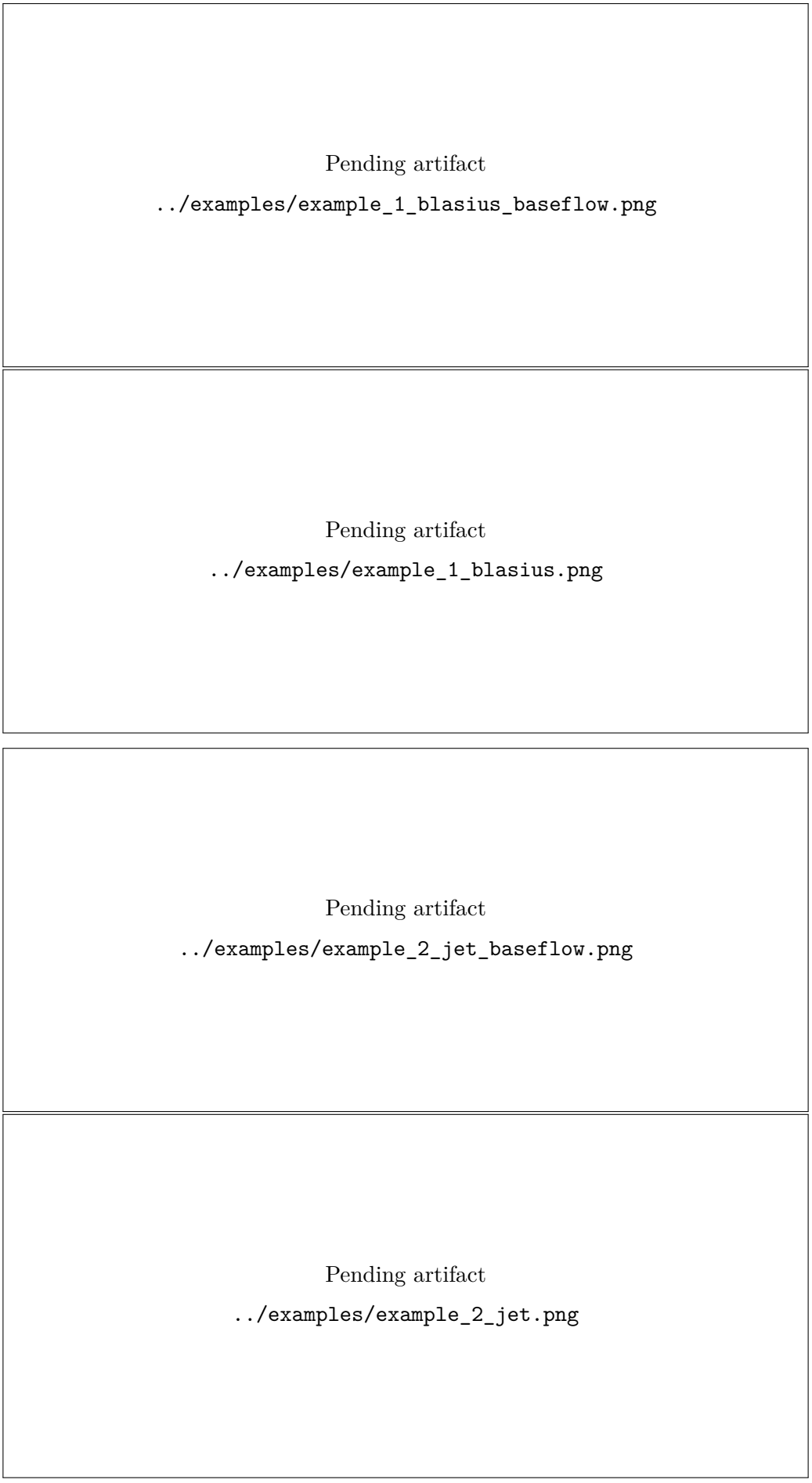


Figure S4: Examples 1 and 2: baseflow and resolvent-summary artifacts for the Blasius boundary layer and the axisymmetric jet.

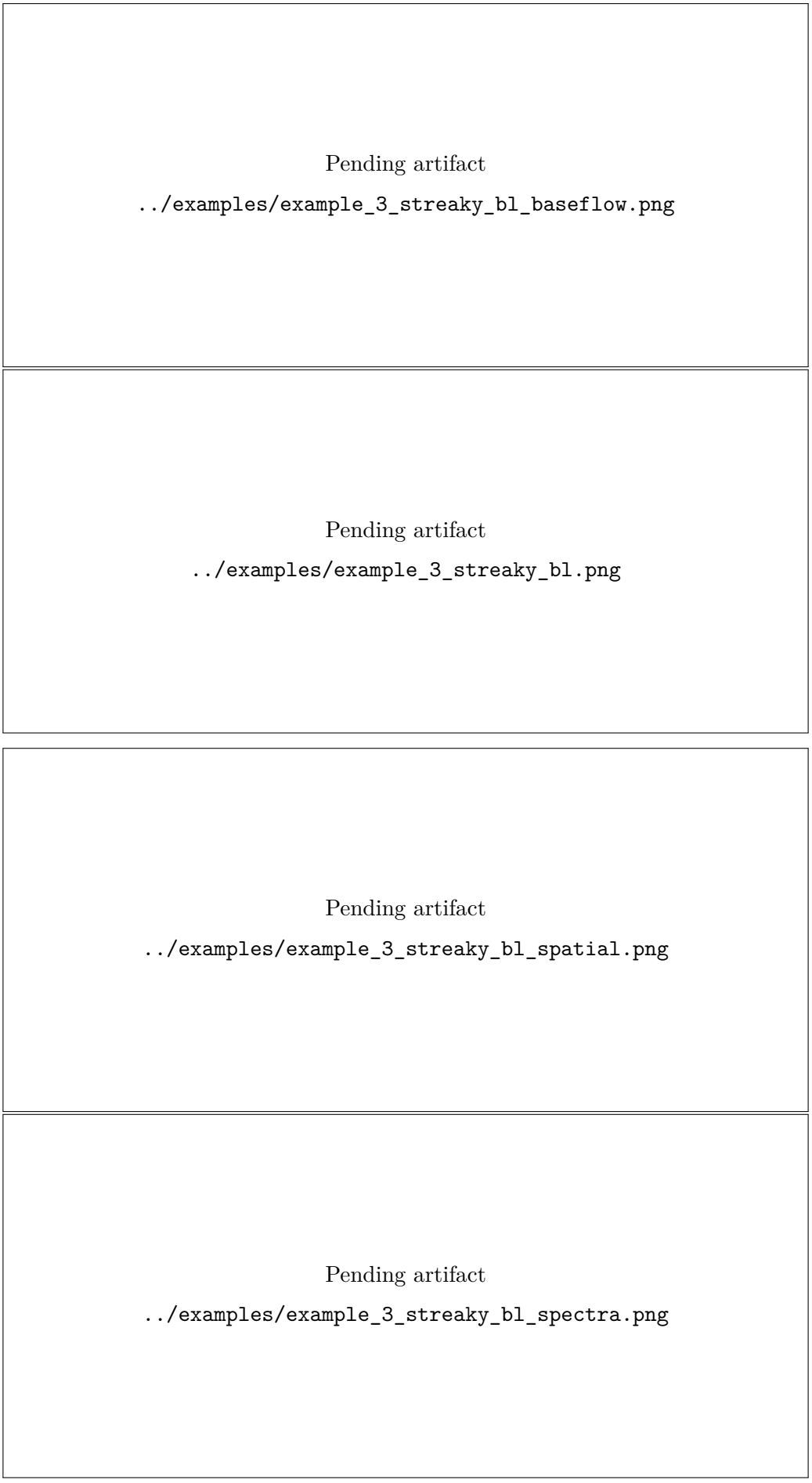
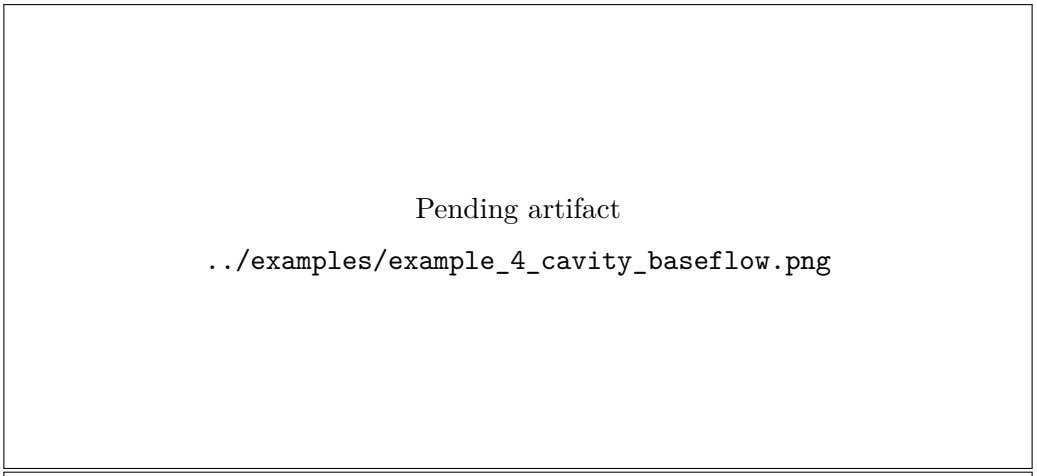
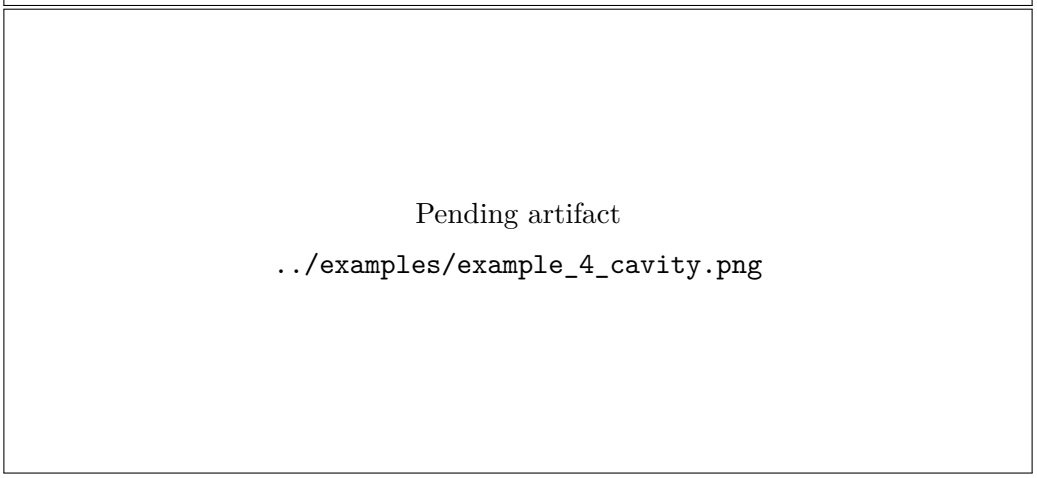
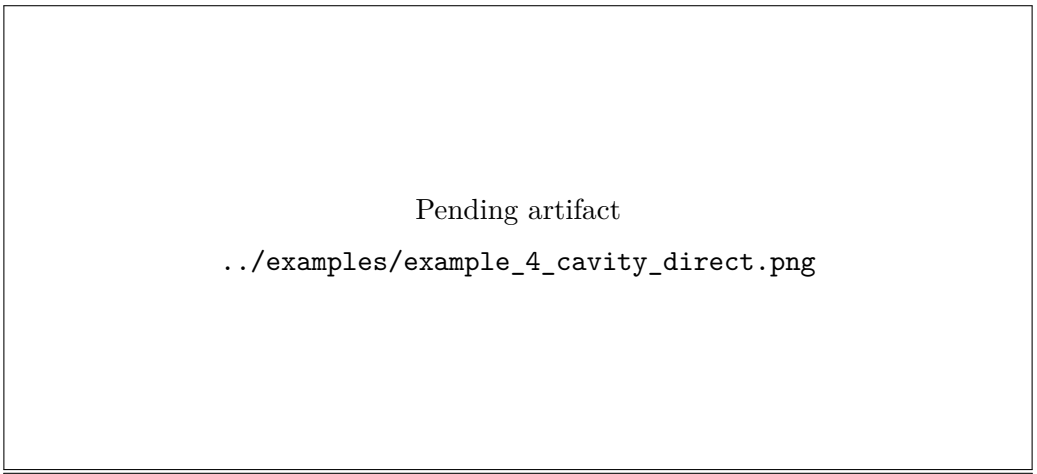


Figure S5: Example 3 streaky-boundary-layer baseflow, temporal mode, spatial mode, and spectral summary.







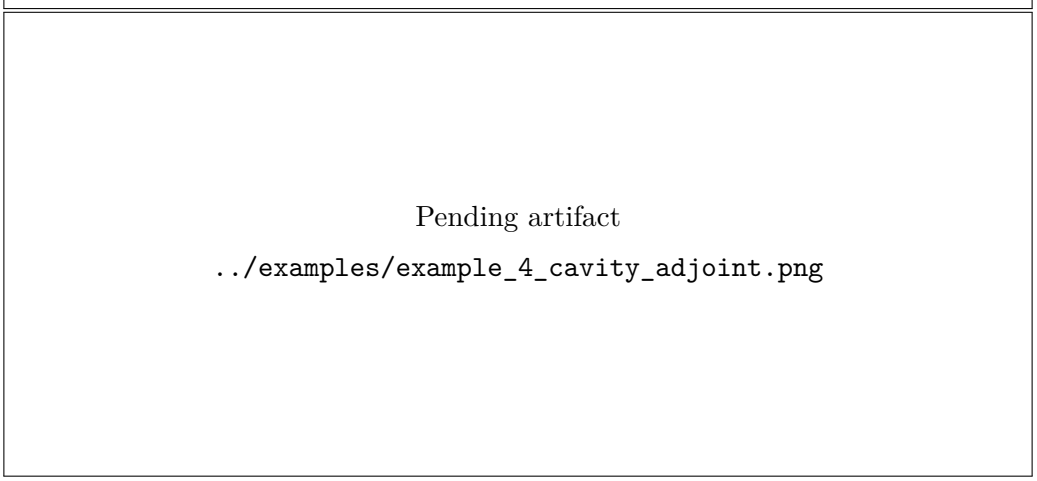
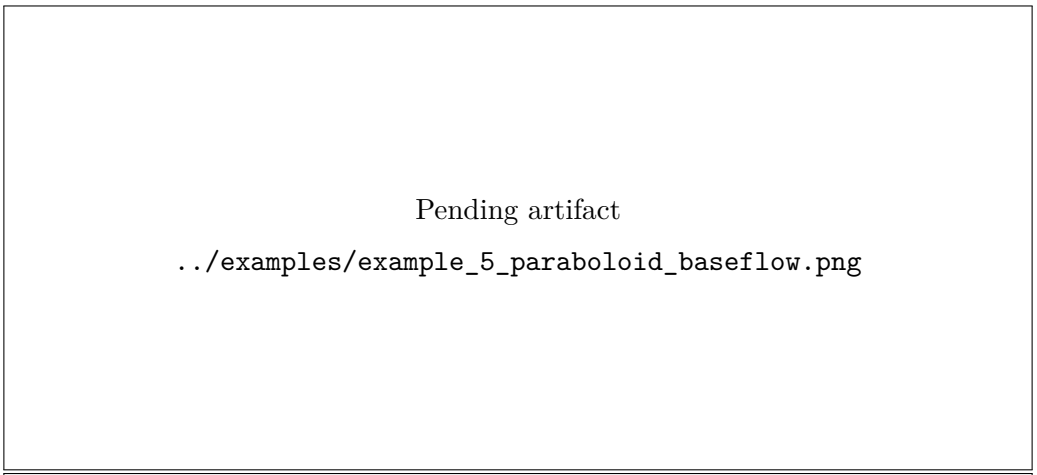


Figure S6: Example 4 cavity baseflow and direct/adjoint global-mode views.



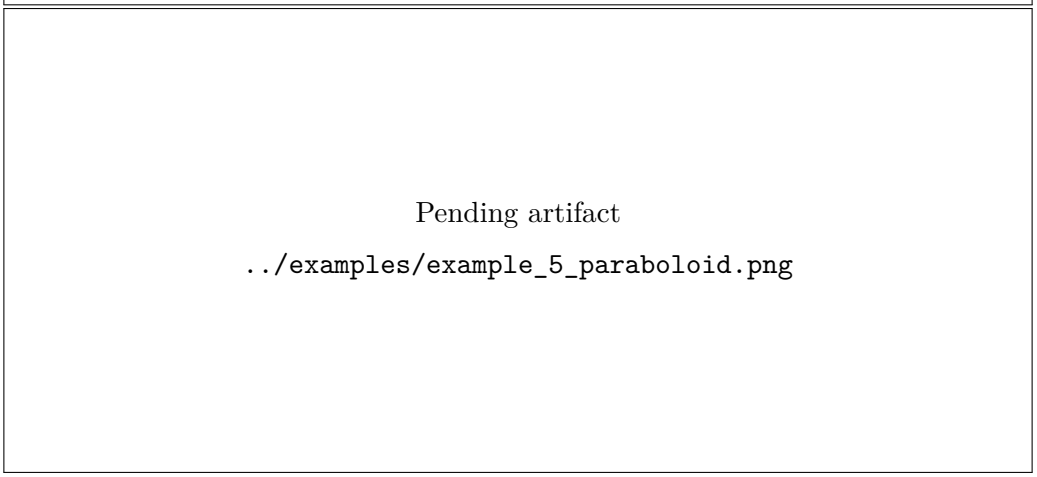


Figure S7: Example 5 paraboloid baseflow and resolvent-summary artifacts.



Figure S8: Robinet/Song oblique SWBLI validation artifact. Left: temporal spectrum with Python (filled), MATLAB (open circles), and Song original (squares) eigenvalues. Right: side-by-side MATLAB (top) and Python (bottom) streamwise velocity eigenmodes for a steady (left column) and oscillatory (right column) branch. Python modes are phase-aligned to MATLAB.

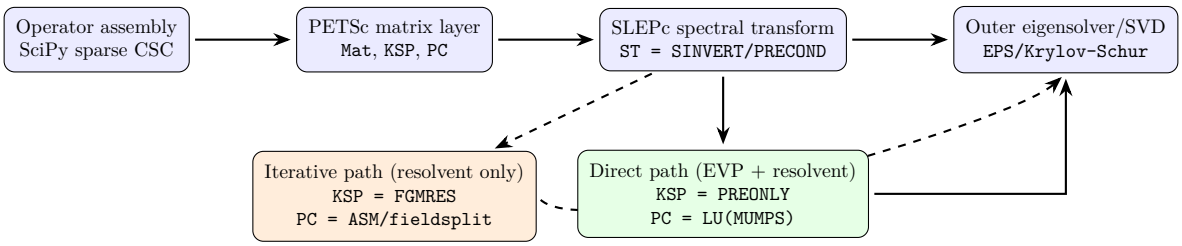


Figure S9: Solver architecture of the current `linstabpy` implementation. The assembled operator is converted to PETSc once, after which the eigensolver and resolvent workflows operate through PETSc/SLEPc.